Specification: Interface Design after Monolith Decomposition during the Migration to Microservices

Alexander Ley
Computer Science Department
University of applied science Bonn-Rhein-Sieg
Sankt Augustin, Germany
s6alleyy@outlook.de

I. Context and Motivation

In software engineering for business applications, the resulting software products are traditionally built as a single deployable unit. This kind of software organization is called a *monolithical architecture*. [1], [2]

Enterprise applications often consist of the following three parts [1]:

1) User Interface (UI)
2) Business Logic Layer (BL)
3) Database (DB)

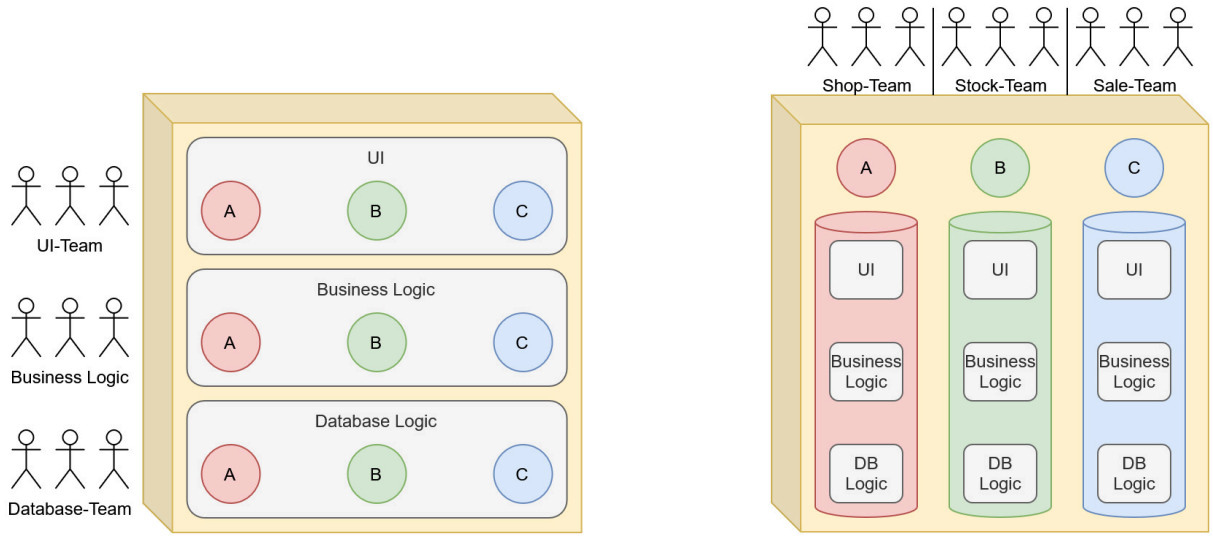
As software is traditionally organized and split around these technical layers the teams maintaining the software are split likewise in the same manner according to these layers leading to "UI Teams", "BL Teams" and "DB Teams". On the left side of Fig. 1, there is an example of a monolith. [1]

Whenever the software has to be adapted all teams may be involved even if the changes are small. According to Conway's Law which is quoted in Quote 1, the communication structure of software teams tends to become mirrored in the software product itself.

> "Any organization that designs a system (defined broadly) will produce a design whose structure is a copy of the organization's communication structure."

Quote 1. Conway's Law quoted by Melvin Conway, 1968 [1]

As consequence of Conway's law, when monoliths grow larger and more complex, they often result in software products which are *highly coupled* and *less cohesive*. *High coupling* means that internal software components generally have many dependencies on other components. *Low cohesion* means that a component does not have a single responsibility and might be responsible for several things. In order words, the software logic is scattered all over the software product. [1], [2]



(a) Monolithical Architecture

(b) Microservice Architecture

Fig. 1. Difference between a *Monolithical Architecture (left)* and a *Microservice Architecture (right)*. Original illustration by C. Nockemann [3].

Therefore, when a monolith grows large, desired qualities such as *scalability*, *maintainability* and *deployability* suffer.

The *microservice architecture*, on the other hand, focuses on addressing the above mentioned downsides of monoliths. As illustrated on the the right side of figure Fig. 1, microservices favor decomposing systems into several, relatively small components which are organized around business capabilities. Each component is implemented as a single deployable unit and can be maintained and scaled individually. [1], [2]

II. Research Problem and Research Questions

Companies that have invested in a monolith might consider migrating and decompose their software into microservices [2]. However, migrating to microservices is a resource-intensive task involving the identification of service boundaries within the monolith and the packaging of identified candidates into self-contained microservices with a defined API [4].

The authors I. Trabelsi, B. Mahmoudi, J. B. Minani, N. Moha, and Y.-G. Guéhéneuc [4] acknowledge that some phases such as the identification of services within a monolith and its decomposition have been extensively studied whereas other phases like the packaging of microservices and generating necessary code for interfaces and APIs are less intensively studied. [4]

A first literature review has so far found and analyzed the *systematic literature reviews (SLRs)* presented in Table I at different levels. Even though the SLR literature might provide some information about the packaging phase, interfaces and API's, it does not focus on them and only touches on these aspects in passing.

Further AI-guided research indicates that no further SLR exists that only focuses on service interfaces and API's after the service identification phase of the migration. Often research might only shortly mention some desired aspects about interfaces but either without a migration background or with a focus on service deployment.

Therefore, this work seeks to conduct a *systematic literature review (SLR)* to shed light on the above-described gap seeking how far research got and finding existing approaches in regard of interface design after the service decomposition step as this is an important step in the migration process. The SLR is trying to answer the following questions:

- RQ1 Which methods and procedures exist for defining service interfaces after the service identification phase in the migration from monoliths to microservices?
  - How are they implemented?
  - Ho mature are they?
- RQ2 Which types, patterns amd metrics are used in the examined methods and procedures?
- RQ3 How are the methods and procedures evaluated?

III. Methodology and Planned Procedures

To examine the above-presented research questions and gain an initial overview of the answers, an iterative, AI-guided[1] creation of search strings was performed to get an overview of the research landscape and to restrict the literature results to the desired topic. The AI was especially useful during the creation of the search strings as it could supply the search string with many alternative keywords, since the researched topic seems to encompass many different kinds of keywords.

The final search string presented in Quote 2 follows the core idea of first searching the abstracts of papers for Monolith, Microservice and Migration keywords and secondly applying a filter using a full-text search for interface aspects.

The presented search string yielded 85 search results on October 7, 2026. To get an initial impression of the amount of

[1]I used DeepSeek AI for all AI-related tasks

TABLE I
The table present the so far analyzed *systematic literature reviews (SLRs)* on different analysis levels. *First Pass* means that at least Abstract and Introduction was read and *Second Pass* means that more than Abstract and Introduction was read such as Conclusion, Discussion, Results or Methods. *Third Pass* means the Paper was read completely.

Author	Title	Year	Depth of Analysis	Focus
I. Trabelsi, B. Mahmoudi, J. B. Minani, N. Moha, and Y.-G. Guéhéneuc [4]	A Systematic Literature Review of Machine Learning Approaches for Migrating Monolithic Systems to Microservices	2025	First Pass	Understand how Machine Learning approaches are used during the Migration from Monoliths to Microservices
Y. Abgaz *et al.* [2]	Decomposition of Monolith Applications Into Microservices Architectures: A Systematic Review	2023	First Pass	Aim to give an overview about the field of Monolith Decomposition. Seeking to give an interrelated picture of various decomposition techniques, benchmarks and metrics.
M. Abdellatif *et al.* [5]	A taxonomy of service identification approaches for legacy software systems modernization	2021	Second Pass	Seeks to give an overview and creates an taxonomy of existing service identification approaches during the migration.
V. Velepucha and P. Flores [6]	Monoliths to microservices - Migration Problems and Challenges: A SMS	2021	Mentioned by other research like [4]	Identification of the problems and challenges associated with the microservice migration.

primary studies dealing with interface aspects, an AI-guided categorization was performed.

[[Abstract: monolith*] OR [Abstract: legacy system*] OR [Abstract: legacy software*] OR [Abstract: legacy application*]]
AND
[[Abstract: microservice*] OR [Abstract: micro-service*] OR [Abstract: micro service*]]
AND
[[Abstract: migrat*] OR [Abstract: decompos*] OR [Abstract: moderniz*] OR [Abstract: refactor*] OR [Abstract: transformation] OR [Abstract: reengineering]]
AND
[[All: service interface*] OR [All: service contract*] OR [All: api design] OR [All: api specification] OR [All: api contract*] OR [All: interface design] OR [All: interface specification] OR [All: interface selection] OR [All: endpoint design] OR [All: service boundar*] OR [All: communication pattern*]]

Quote 2. Final Search-String of the iteration procedure.

The AI categorized 25 of the 85 papers as primary studies that directly address interface aspects and a further 28 as borderline cases that cover interface aspects only in passing.

To sum up, at first glance there might be enough literature for an SLR. The next steps would include obtaining a more detailed view of (SLR-)literature focusing on the migration phases after the decomposition step. Based on the insights, the search string might be adapted.

In general, this work plans to follow the SLR-procedure proposed by B. Kitchenham, O. Pearl Brereton, D. Budgen, M. Turner, J. Bailey, and S. Linkman [7].

Acknowledgement

During the work DeepSeek AI was used to guide the literature review, create search strings and categorize the first 85 results to get first insights.

References

[1] J. Lewis and M. Fowler, "Microservices: A Definition of This New Architectural Term." Accessed: Aug. 25, 2026. [Online]. Available: https://martinfowler.com/articles/microservices.html

[2] Y. Abgaz *et al.*, "Decomposition of Monolith Applications Into Microservices Architectures: A Systematic Review," *IEEE Transactions on Software Engineering*, vol. 49, no. 8, pp. 4213–4242, Aug. 2023, doi: 10.1109/TSE.2023.3287297.

[3] C. Nockemann, "Domain-Driven Design in Der Praxis," in *Presentations during the Lecture of Object-Oriented Component Architectures (OOKA) Renamed to Service-Oriented Component Architectures (SEKA)*, University of applied science Bonn-Rhein-Sieg, 2025.

[4] I. Trabelsi, B. Mahmoudi, J. B. Minani, N. Moha, and Y.-G. Guéhéneuc, "A Systematic Literature Review of Machine Learning Approaches for Migrating Monolithic Systems to Microservices," *IEEE Transactions on Software Engineering*, vol. 51, no. 11, pp. 2972–2995, Nov. 2025, doi: 10.1109/TSE.2025.3603897.

[5] M. Abdellatif *et al.*, "A Taxonomy of Service Identification Approaches for Legacy Software Systems Modernization," *Journal of Systems and Software*, vol. 173, p. 110868, Mar. 2021, doi: 10.1016/j.jss.2020.110868.

[6] V. Velepucha and P. Flores, "Monoliths to Microservices - Migration Problems and Challenges: A SMS," in *Proceedings of the Second International Conference on Information Systems and Software Technologies (ICI2ST 2021)*, Quito, Ecuador: IEEE, Mar. 2021, pp. 135–142. doi: 10.1109/ICI2ST51859.2021.00027.

[7] B. Kitchenham, O. Pearl Brereton, D. Budgen, M. Turner, J. Bailey, and S. Linkman, "Systematic Literature Reviews in Software Engineering – A Systematic Literature Review," Jan. 2009. doi: 10.1016/j.infsof.2008.09.009.